simulation_of_rv



驻波的数理统计老师简直是混沌邪恶派，上课梦到哪句说哪句，作业也布置不清楚，考试还创新地说要手写代码模拟分布。所以今天就把一些常见分布的模拟思路和代码总结一下。



常见方法

最常用的方法是基于下面这个定理的，从均匀分布出发，生成服从指定分布的数据。

Thm 1: 逆变换法

> 设随机变量$U \sim U(0,1)$，F是希望模拟的随机变量X的cdf，则随机变量$X = F^{-1}(U)$服从目标分布。

Proof:

$$P(X \leq x) = P(F^{-1}(U) \leq x) = P(U \leq F(x)) = F(x) \qquad \square$$

Rmk: 对于离散分布，逆变换法使用广义逆函数：

$$X = F^{-1}(U) = \min\{k : F(k) \geq U\}.$$

通俗理解来说，就是找到第一个使得F(k)大于U的k。

对于一些复杂的连续分布，还有一种方法是acceptence-rejection method，它的原理是基于Monte - Carlo。

先把算法展示如下：
Algorithm: Acceptance-Rejection Sampling

Input:

- Target density function $f(x)$
- Proposal distribution G with density $g(x)$
- Constant c satisfying

$$f(x) \le c\, g(x) \quad \forall x \in \mathbb{R}$$

- Number of desired samples N

Output:

- Samples $X_1, \ldots, X_N$ approximately distributed according to $f(x)$

Procedure

1. Set $i \leftarrow 1$.
2. While $i \le N$:
   i. Generate a proposal sample $Y \sim G$.
   ii. Generate $U \sim \mathrm{Uniform}(0,1)$.
   iii. Compute the acceptance probability
   $$p(Y) = \frac{f(Y)}{c\, g(Y)}.$$
   iv. If
   $$U \le p(Y),$$
   accept the proposal and set
   $$X_i \leftarrow Y.$$
   Then set $i \leftarrow i + 1$.
   v. Otherwise, reject Y and return to Step 2.i.
3. Return $X_1, \ldots, X_N$.

这个用来辅助的分布G可以是任意的好算的分布，比如均匀分布或者高斯分布。直观上来说，就是对于一个在$cg(x)$下、x轴上方的区域进行随机采样，如果采样点满足不等式条件（几何意义上就是落在目标分布的pdf $f(x)$下方），就接受这个点，否则就拒绝。

接下来是有效性证明。

Theorem 2: Effectiveness of the acceptance-rejection method

> Let f and g be probability density functions on $\mathbb{R}$. Suppose that there exists a constant $c \ge 1$ such that

$$f(x) \le c\, g(x) \quad \forall x \in \mathbb{R}$$

Generate two independent random variables

$$Y \sim g, \qquad U \sim \mathrm{Uniform}(0,1).$$

Accept Y if

$$U \le \frac{f(Y)}{c\, g(Y)}.$$

Let $B = \left\{ U \le \frac{f(Y)}{c\, g(Y)} \right\}$ denote the acceptance event. Then:

1. The probability of accepting a proposal is

$$P(B) = \frac{1}{c}.$$

2. If N denotes the number of proposals required to obtain the first accepted sample, then

$$N \sim \mathrm{Geo}\left(\frac{1}{c}\right), \quad E[N] = c.$$

3. The conditional distribution of Y given acceptance is the target distribution f. In other words,

$$Y \mid B \sim f.$$

Proof:
先证明接受概率$P(B) = \frac{1}{c}$，由B的定义：

$$P(B|Y=t) = \frac{f(t)}{cg(t)}.$$

因此：

$$P(B) = \int_{-\infty}^{\infty} P(B|Y=t)g(t)dt = \int_{-\infty}^{\infty} \frac{f(t)}{cg(t)}g(t)dt = \frac{1}{c}\int_{-\infty}^{\infty} f(t)dt = \frac{1}{c}.$$

那么因为N表示第一次接受样本所需的尝试次数，所以$N \sim \mathrm{Geo}\left(\frac{1}{c}\right)$，$E[N] = c$。
最后证明条件分布$Y|B \sim f$，由条件概率公式：

$$P(Y \le x|B) = \frac{P(Y \le x, B)}{P(B)} = c\int_{-\infty}^{x} \frac{f(t)}{cg(t)} g(t)dt = \int_{-\infty}^{x} f(t)dt.$$

因此，$Y|B \sim f$。□

方法说完了，但是实现上python的Numpy库已经封装好了很多常见分布的函数，在具体代码的地方会进行说明。（不过Numpy有的函数的逻辑也是逆变换法）

接下来进行一些初始化为了行文方便：

```
rng = np.random.default_rng(42)  # 初始化随机数生成器
N = 10000 # 初始化10000个样本
```

Bernoulli distribution

这我真不知道有什么意义啊。
给出两种想法吧，第一种是直接使用numpy内置函数，然后模拟N次伯努利实验，由LLN保证它会收敛到p。

```
p_bern = 0.3
bernoulli_data = rng.binomial(n=1, p=p_bern, size=N)
```

这其实就模拟了N次伯努利实验了，绘图部分大同小异，就这里展示一下，其他的可以在github上看。

```
# 1. 伯努利分布
axes[0].hist(
    bernoulli_data,
    bins=np.arange(-0.5, 2.5, 1),
    weights=np.ones(N) / N,
    color="crimson",
    edgecolor="white",
    alpha=0.65,
    label="Sample histogram",
)

# 理论概率：P(X=1)=p，P(X=0)=1-p
k_bern = np.array([0, 1])
pmf_bern = np.array([1 - p_bern, p_bern])

axes[0].plot(
    k_bern, pmf_bern, "r-o",
    linewidth=2,
    markersize=5,
    label="Theoretical PMF",
)

axes[0].set(
    title=f"Bernoulli: p = {p_bern}",
    xlabel="Value",
    ylabel="Probability",
    xlim=(-0.5, 1.5),
)
axes[0].set_xticks([0, 1])
```

第二种采用逆变换法

```
# 伯努利分布
# 分布函数F(0) = 1-p，F(1) = 1，因此只要判别随机数是否大于1-p即可
p_bern = 0.3
bernoulli_u = rng.random(N)
bernoulli_data = (bernoulli_u > 1 - p_bern).astype(int)
```

Uniform distribution

我也不懂这有什么意义

```
# 均匀分布
a = 0
b = 1
uniform_data = rng.uniform(a, b, size=N)
```

逆变换法

```
# 均匀分布
# F^{-1}(u) = a + (b-a)u
a = 0
b = 1
uniform_u = rng.random(N)
uniform_data = a + (b - a) * uniform_u
```

Exponential distribution

```
# 指数分布
lam = 2
exponential_data = rng.exponential(scale=1 / lam, size=N)
```

这里稍微注意一下scale参数是$\frac{1}{\lambda}$，最大的难点是不要敲错（

逆变换法就是求出来它的反函数：

```
# 指数分布
lam = 2
# 理论密度：f(x) = lam * exp(-lam*x)，x >= 0
# CDF：F(x) = 1 - exp(-lam*x)，x >= 0
# 逆变换：F^{-1}(u) = -log(1-u)/lam
exponential_u = rng.random(N)
exponential_data = -np.log1p(-exponential_u) / lam
```

Binomial distribution

事实上我们在Bernoulli分布中已经模拟了二项分布的一个$n = 1$的特例了，而且我们当时也用的是binomial函数。

```
# 二项分布
n = 10
p = 0.4
binomial_data = rng.binomial(n=n, p=p, size=N)
```

而逆变换法的话，强大的numpy库可以让我们快速计算离散的分布函数：

```
def inverse_transform_discrete(u, support, pmf):
    """离散分布的逆变换法：X = min{k: F(k) >= U}."""
    cdf = np.cumsum(pmf)
    # searchsorted 找到第一个满足 F(k) >= u 的位置
    indices = np.searchsorted(cdf, u, side="left")
    return support[indices]
```

Rmk：ai写的代码比较“具有健壮性”，驻波这里就直接把核心的部分给提出来了，完整的包含各种检查的在github可以看

最后，我们利用这个函数，针对二项分布进行实现：

```
# 二项分布
n = 10
p = 0.4
# 理论概率：P(X=k) = C(n,k) * p^k * (1-p)^(n-k)
# CDF：F(k) = sum_{j=0}^k C(n,j) * p^j * (1-p)^(n-j)
# 先计算 PMF，再根据离散分布的 CDF 使用逆变换法
binomial_u = rng.random(N)
binomial_support = np.arange(n + 1)
binomial_pmf = np.array([
    comb(n, k) * p**k * (1 - p)**(n - k)
    for k in binomial_support
])
binomial_data = inverse_transform_discrete(
    binomial_u,
    binomial_support,
    binomial_pmf,
)
```

Negative Binomial distribution

```
# 负二项分布
r = 5       # 需要成功 r 次
p_nb = 0.5 # 每次试验的成功概率
negative_binomial_data = rng.negative_binomial(n=r, p=p_nb, size=N)
```

这种方法不多解释。

逆变换法的话，主要是先实现一下负二项分布的pmf。负二项分布的含义是：做独立伯努利试验，每次成功概率为 p，直到累计成功 r 次。X 表示在这 r 次成功之前失败的次数。则：

$$P(X=k)=\binom{k+r-1}{k}p^r(1-p)^k,\quad k=0,1,2,\dots$$

可以看到这里有组合数和幂，所以我们采用递推的方式来实现。首先$X=0$时，$P(X=0)=p^r$，然后我们可以利用递推公式：

$$\frac{P(X=k)}{P(X=k-1)}=\frac{k+r-1}{k}(1-p)$$

```
def negative_binomial_pmf_until(r, p, largest_u):
    """递推负二项分布的 PMF，直到累计概率覆盖本批次的 U。"""
    pmf = [p**r]
    total = pmf[0]
    k = 0

    while total < largest_u:
        k += 1
        # P(X=k) / P(X=k-1) = ((k+r-1)/k) * (1-p)
        next_probability = pmf[-1] * (k + r - 1) / k * (1 - p)
        # pmf[-1]是取这个列表的最后一个值，也就是P(X=k-1)
        pmf.append(next_probability)
        total += next_probability

    return np.asarray(pmf, dtype=float)
```

接下来我们再调用之前的函数：

```
# 负二项分布
# X 表示第 r 次成功之前的失败次数
r = 5       # 需要成功 r 次
p_nb = 0.5 # 每次试验的成功概率
# 理论概率：P(X=k) = C(k+r-1,k) * p_nb^r * (1-p_nb)^k
# pmf：F(k) = sum_{j=0}^k C(j+r-1,j) * p_nb^r * (1-p_nb)^j
negative_binomial_u = rng.random(N)
negative_binomial_pmf = negative_binomial_pmf_until(
    r,
    p_nb,
    largest_u=float(negative_binomial_u.max()),
)
negative_binomial_support = np.arange(negative_binomial_pmf.size)
negative_binomial_data = inverse_transform_discrete(
    negative_binomial_u,
    negative_binomial_support,
    negative_binomial_pmf,
)
```

可以看到代码结构和二项分布的实现是类似的，只是pmf的计算方式不同，我们用了个函数来实现它。

Poisson distribution

```
# 泊松分布
lam_pois = 5   # 泊松分布的参数 lambda
poisson_data = rng.poisson(lam=lam_pois, size=N)
```

伟大的numpy库又帮我们封装好了。

逆变换法的话，因为Poisson分布的PMF是：

$$P(X = k) = \frac{e^{-\lambda}\lambda^k}{k!}, \quad k = 0, 1, 2, \ldots$$

我们还是用递推来实现：

```python
def poisson_pmf_until(lam, largest_u):
    """递推泊松分布的 PMF，直到累计概率覆盖本批次的 U。"""
    pmf = [np.exp(-lam)]
    total = pmf[0]
    k = 0

    while total < largest_u:
        k += 1
        # P(X=k) = P(X=k-1) * lambda / k
        next_probability = pmf[-1] * lam / k
        pmf.append(next_probability)
        total += next_probability

    # 省略的尾部概率小于本批次均匀随机数所需覆盖的范围
    return np.asarray(pmf, dtype=float)


# 泊松分布
lam_pois = 5   # 泊松分布的参数 lambda
# 理论概率：P(X=k) = lam_pois^k * exp(-lam_pois) / k!
# CDF：F(k) = sum_{j=0}^k lam_pois^j * exp(-lam_pois) / j!
# 先递推 PMF，再根据离散分布的 CDF 使用逆变换法
poisson_u = rng.random(N)
poisson_pmf_values = poisson_pmf_until(
    lam_pois,
    largest_u=float(poisson_u.max()),
)
poisson_support = np.arange(poisson_pmf_values.size)
poisson_data = inverse_transform_discrete(
    poisson_u,
    poisson_support,
    poisson_pmf_values,
)
```

这些有了前面的铺垫都不难理解

Normal distribution

```
# 正态分布：得到Y~N(mu_normal, sigma_normal^2)。
mu_normal = 0
sigma_normal = 1
normal_data = rng.normal(loc=mu_normal, scale=sigma_normal, size=N)
```

关于逆变换法，有两种思路，第一种还是去找反函数，但是人家CDF都没有数值解，所以这一过程变成用数值积分进行：

$$\Phi(x) = \int_{-\infty}^{x} \phi(t)\, dt$$

然后通过 Newton 迭代求解

$$\Phi(x) = u$$

迭代公式为

$$x_{k+1} = x_k - \frac{\Phi(x_k) - u}{\phi(x_k)}.$$

这一部分代码量也比较大，实际计算时间复杂度也高，就不多加介绍了，代码实现也在github里面。

第二种方法是使用 Box-Muller transform，它的原理是利用两个独立的均匀分布随机变量生成两个独立的标准正态分布随机变量。

Theorem 3: Box-Muller Transform

> Let $U_1, U_2 \overset{\text{i.i.d.}}{\sim} U(0,1)$, Set $R = \sqrt{-2\ln U_1}$ and $\Theta = 2\pi U_2$. Then the random variables
>
> $$Z_0 = R\cos(\Theta) = \sqrt{-2\ln U_1}\cos(2\pi U_2)$$
> $$Z_1 = R\sin(\Theta) = \sqrt{-2\ln U_1}\sin(2\pi U_2)$$
>
> are independent and each has a standard normal distribution. That is, $Z_0, Z_1 \overset{\text{i.i.d.}}{\sim} N(0,1)$.

Proof:
不难注意到$R = \sqrt{Z_0^2 + Z_1^2}$，$\Theta = \arctan(Z_1/Z_0)$，所以$(Z_0, Z_1)$和$(R, \Theta)$之间存在一个极坐标变换，Jacobian行列式为：

$$|J| = \begin{vmatrix} \frac{\partial Z_0}{\partial R} & \frac{\partial Z_0}{\partial \Theta} \\ \frac{\partial Z_1}{\partial R} & \frac{\partial Z_1}{\partial \Theta} \end{vmatrix} = \begin{vmatrix} \cos(\Theta) & -R\sin(\Theta) \\ \sin(\Theta) & R\cos(\Theta) \end{vmatrix} = R.$$

我们的思路是先把(R, Θ)的联合分布求出来，然后再通过Jacobian变换得到(Z_0, Z_1)的联合分布。
先求 R 的分布。对 $r > 0$，

$$F_R(r) = P(R \le r) = P\left(\sqrt{-2\ln U_1} \le r\right) = P\left(-2\ln U_1 \le r^2\right).$$

因为 $U_1 \in (0,1)$，

$$-2\ln U_1 \le r^2 \iff \ln U_1 \ge -\frac{r^2}{2} \iff U_1 \ge e^{-r^2/2}.$$

因此

$$F_R(r) = P\left(U_1 \ge e^{-r^2/2}\right) = 1 - e^{-r^2/2}, \qquad r > 0.$$

求导得 R 的密度函数

$$f_R(r) = r e^{-r^2/2}, \qquad r > 0.$$

Θ 更好求，$U_2 \sim U(0,1)$，所以

$$\Theta \sim U(0, 2\pi),$$

其密度为

$$f_\Theta(\theta) = \frac{1}{2\pi}, \qquad 0 < \theta < 2\pi.$$

由 R 与 Θ 独立，得到 (R, Θ) 的联合密度

$$f_{R,\Theta}(r,\theta) = f_R(r) f_\Theta(\theta) = \frac{1}{2\pi} r e^{-r^2/2}, \qquad r > 0,\ 0 < \theta < 2\pi.$$

把最开始我们算出来的逆变换和Jacobian行列式代入联合密度公式：

$$f_{Z_0,Z_1}(z_0,z_1)=\frac{1}{2\pi}e^{-(z_0^2+z_1^2)/2}=\left(\frac{1}{\sqrt{2\pi}}e^{-z_0^2/2}\right)\left(\frac{1}{\sqrt{2\pi}}e^{-z_1^2/2}\right)$$

所以$Z_0,Z_1\overset{\text{i.i.d.}}{\sim}N(0,1)$ □

```
def normal_box_muller(n, rng):
    """Generate standard normal samples with the Box-Muller transform."""
    pair_count = (n + 1) // 2   # 因为这个方法生成的是“一对正态分布”，所以需要的对数是 n/2，向上取整
    u1 = np.clip(rng.random(pair_count), np.finfo(float).tiny, 1.0)
    u2 = rng.random(pair_count)

    radius = np.sqrt(-2.0 * np.log(u1))
    angle = 2.0 * np.pi * u2
    z1 = radius * np.cos(angle)
    z2 = radius * np.sin(angle)

    return np.concatenate((z1, z2))[:n]
```

Gamma distribution

Γ分布我们采用acceptance-rejection method来模拟，目标$\Gamma(\alpha,\beta)$的pdf为：

$$f(y)=\frac{y^{\alpha-1}e^{-y/\beta}}{\Gamma(\alpha)\beta^{\alpha}}$$

用于辅助的proposal分布我们选择$Exp(\lambda)$，它的pdf为：

$$g(y)=\lambda e^{-\lambda y}$$

那么接受概率就是：

$$p=\frac{f(y)}{c\cdot g(y)}=\frac{y^{\alpha-1}e^{-y/\beta}}{\Gamma(\alpha)\beta^{\alpha}}\cdot\frac{1}{c\cdot\lambda e^{-\lambda y}}=\frac{y^{\alpha-1}e^{-(1/\beta-\lambda)y}}{\Gamma(\alpha)\beta^{\alpha}\lambda c}$$

令：

$$\delta=\frac{1}{\beta}-\lambda$$

则：

$$p=\frac{y^{\alpha-1}e^{-\delta y}}{\Gamma(\alpha)\beta^{\alpha}\lambda c}$$

这就是下面代码的主要逻辑。第一个函数是Γ分布的pdf，第二个函数是acceptance-rejection method的实现。

```
# Target Gamma density
def gamma_density(x):
    x = np.asarray(x)
    return np.where(
        x >= 0,
        x ** (alpha - 1)
        * np.exp(-x / theta)
        / (math.gamma(alpha) * theta ** alpha),
        0.0
    )

# Acceptance-Rejection sampling
def gamma_accept_reject(n, seed=1234):
    rng = np.random.default_rng(seed)

    samples = []
    total_proposals = 0
    total_accepted = 0

    while len(samples) < n:
        batch_size = max(1000, n - len(samples))

        # Generate proposal samples from Exp(rate=0.25)
        y = rng.exponential(scale=proposal_scale, size=batch_size)

        # Generate U(0,1)
        u = rng.random(batch_size)

        # Calculate f(y) / (c * g(y))
        accept_prob = (
            y ** (alpha - 1)
            * np.exp(-delta * y)
            / (
                math.gamma(alpha)
                * theta ** alpha
                * lam
                * c
            )
        )

        accept = u <= accept_prob

        samples.extend(y[accept].tolist())
```

```
        total_proposals += batch_size
        total_accepted += np.sum(accept)

    samples = np.asarray(samples[:n])
    acceptance_rate = total_accepted / total_proposals

    return samples, acceptance_rate
```

Chi-squared distribution

这是Gamma分布的一个特例，$\chi^2(k) \sim \Gamma(k/2, 2)$，所以略过

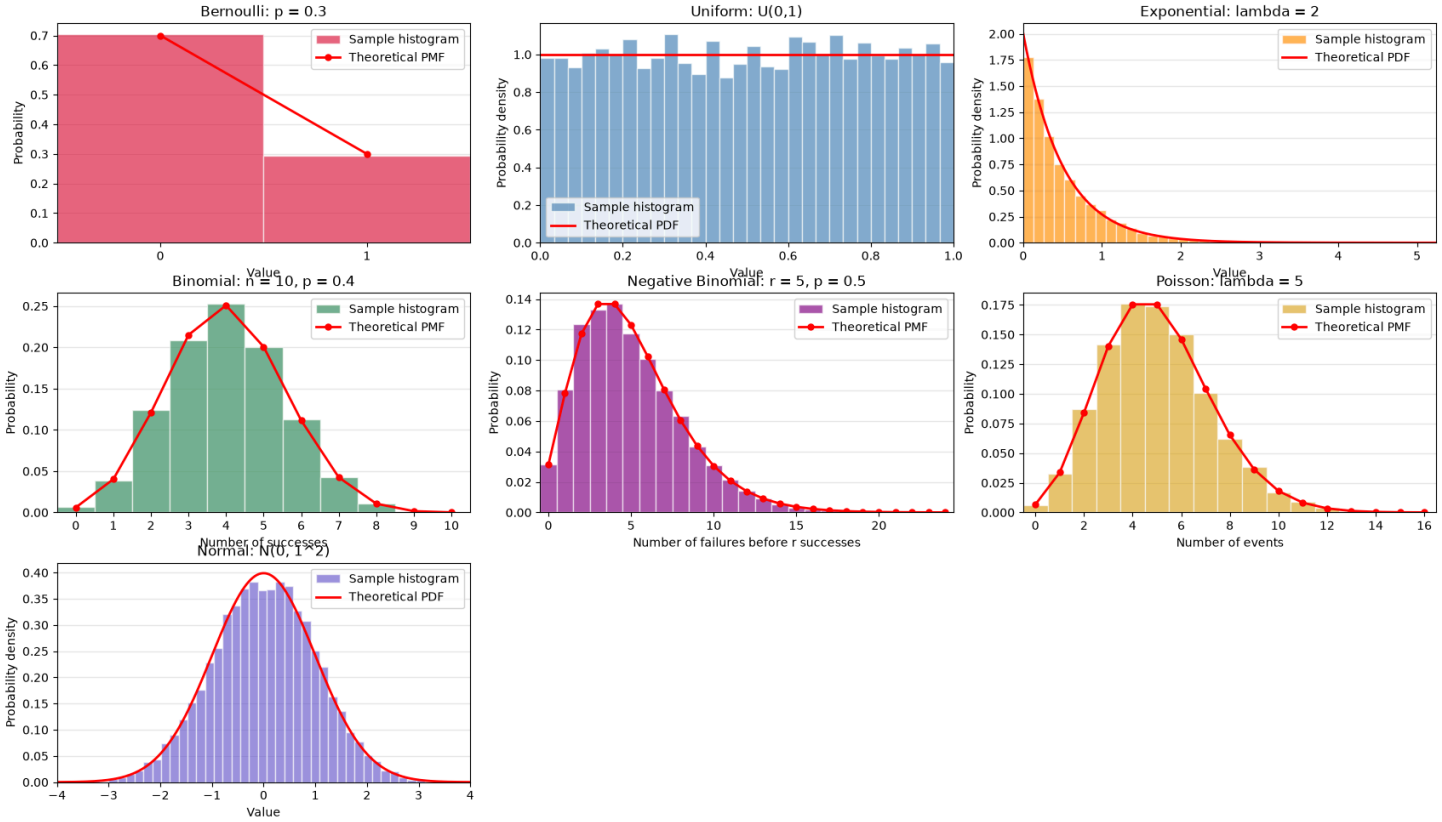
Beta distribution

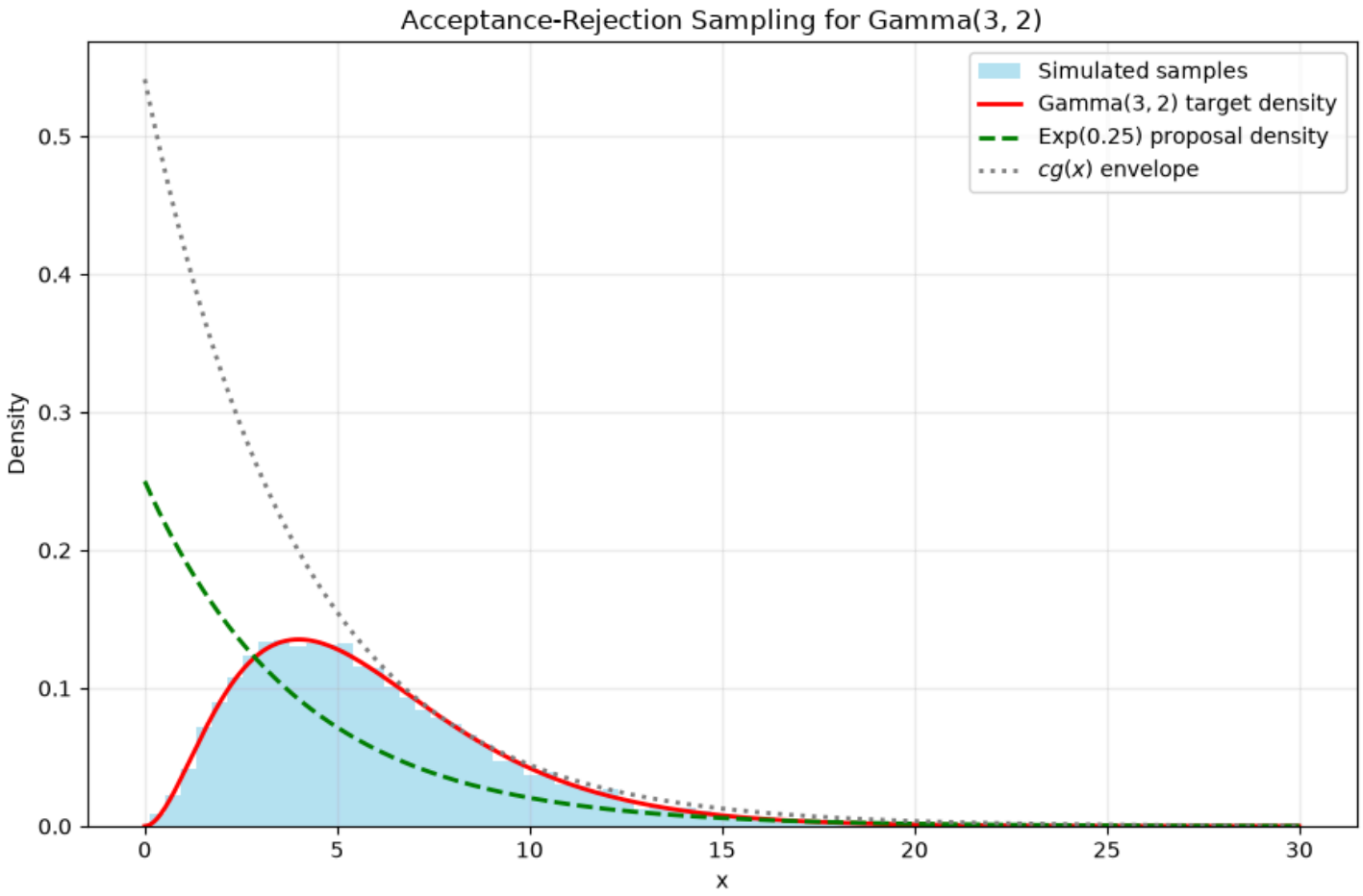
哈哈，不想写



Conclusion

各种分布的图


Bernoulli: p = 0.3
Uniform: U(0,1)
Exponential: lambda = 2
Binomial: n = 10, p = 0.4
Negative Binomial: r = 5, p = 0.5
Poisson: lambda = 5
Normal: N(0, 1^2)
Sample histogram
Theoretical PMF
Theoretical PDF
Probability
Probability density
Value
Number of successes
Number of failures before r successes
Number of events


Acceptance-Rejection Sampling for Gamma(3, 2)
Simulated samples
Gamma(3, 2) target density
Exp(0.25) proposal density
cg(x) envelope
Density
x